Week 6 — Research Lab Handouts

Software Measurement

Software Engineering • Atlantic Technological University (ATU) Donegal

Handout A: OrderCalculator Code Sample

CODE SAMPLE

The short function below calculates a shipping charge for an order. You do not need to run or compile it; this is a reading exercise. Read it carefully for Task 1.

```
public double calcShip(double wt, String dest, boolean p, int q) {
    double c = 0;
    if (dest.equals("LOCAL")) {
        if (wt < 1) {
            c = 2.50;
        } else if (wt < 5) {
            c = 4.00;
        } else {
            c = 4.00 + (wt - 5) * 0.50;
        }
    } else if (dest.equals("NATIONAL")) {
        if (wt < 1) {
            c = 5.00;
        } else if (wt < 5) {
            c = 8.00;
        } else {
            c = 8.00 + (wt - 5) * 1.00;
        }
    } else {
        if (wt < 1) {
            c = 12.00;
        } else if (wt < 5) {
            c = 20.00;
        } else {
            c = 20.00 + (wt - 5) * 2.50;
        }
    }
    if (p) {
        c = c * 0.5;
    }
    if (q > 10) {
        c = c * 0.9;
    }
    return c;
}
```

What to look for

- Count the decision points (every if / else if) to estimate this method's cyclomatic complexity by hand. How many independent paths can execution take through it?
- Notice the three destination branches (LOCAL / NATIONAL / international) each repeat the same 'three weight tiers' pattern: a duplication smell, even though the exact numbers differ.
- The parameter and variable names (wt, dest, p, q, c) give no clue what they mean without reading the whole method: a naming and readability concern.
- There is no handling shown for invalid input (e.g., negative weight): worth noting as a robustness and reliability concern even though it is not a 'complexity' metric as such.